

Heuristic-primary, replay-grounded reranking for Pokémon TCG agents

A safe-by-construction hybrid that took a heuristic ceiling of about 735 to a live peak of 924.2, and the honest story of five shortcuts that didn't work

K S Vijayaraaghavan

The problem, and why heuristics alone stall out

Pokémon TCG combines hidden information (the opponent's hand and deck) with a large space of legal plays each turn, which is exactly what the competition's own framing calls out as the core difficulty. A hand-tuned, rule-based heuristic is a reasonable place to start: it encodes real domain knowledge (energy sequencing, KO math, retreat safety) as explicit rules you can read and debug. But every deck I built this way, three main archetypes and eleven abandoned ones, converged on the same ceiling: roughly 690 to 735 live rating. Other public teams reported the same wall on rule-based-plus-search designs, which told me this wasn't a tuning problem. Deeper search (4-ply minimax, up to 30 determinizations) and a genetic-algorithm weight search both landed at statistical parity with hand-picked weights. More of the same lever wasn't going to move the number.

What failed, and why it's worth reporting

Public teams were reaching 800 to 1034 on the same archetypes with behavior cloning (BC) on real ladder replay data, so that was the obvious next lever. My first attempt was to train a classifier to imitate winning players and let it drive the agent directly. That failed, decisively, five separate times, across genuinely different setups: more training data (61, then 340, then 83 deck-specific games), added lethal/KO features by hand, an ensemble-voting variant, and a version conditioned on turn history. Every one of them scored between 2% and 26% locally against my own proven heuristic, no matter how offline classification accuracy improved (52.6% up to 63.5%). Offline accuracy was telling me the wrong thing.

The diagnosis held up under scrutiny: a per-decision classifier has no way to represent a plan that spans several decisions in one turn, such as which attacker to save energy for or where to place damage counters toward a three-turn kill. My heuristic tracks that sequencing explicitly through a plan object that every sub-decision consults. A pointwise BC model re-derives each decision on its own and quietly breaks the plan's internal consistency, even when any single decision looks fine in isolation. A related "plan-then-subordinate" turn architecture (a technique one public team reported success with) and a learned turn-value model both landed at flat parity too: safe, but not a proven gain. That points at a narrower conclusion than "BC doesn't work here." The value seems to live specifically in constraining BC to places where sequencing risk is low, not in making the BC model itself bigger or better-informed.

I'm reporting this as a real result rather than a wasted week, because it's the reason the final design is conservative by architecture instead of "BC bolted on top."

The design that worked: heuristic first, BC only on close calls

The heuristic always computes its full ranking of legal options first. A GBDT model trained on 340 real top-player ladder replays (later retrained deck-specifically on 72 to 83 games) only gets consulted when two things are both true: the heuristic's top options are within a small score margin of each other, and the decision type sits on an explicit allowlist of contexts I judged safe to override, single-pick decisions that don't touch the plan. Outside that gate, or if the reranker fails in any way, the heuristic's own top pick plays untouched. That makes the design safe by construction. It can't play worse than the underlying heuristic, because of how it's built, not because I hoped so.

Locally this is almost invisible: parity or a few points either way against the plain heuristic, across thousands of test games. Live, the same submission (ref 55464921) debuted at 796.9 and eventually peaked at 924.2, a jump of nearly 200 points over the heuristic ceiling and the single biggest result of the project. That gap became the project's central lesson: local win-rate testing kept turning out to be an unreliable predictor of live ladder performance, in both directions (the failed BC-as-primary attempts showed good local signal predicting nothing, and this build showed the reverse). I stopped treating that as noise to explain away. Reading the competition's own evaluation documentation confirmed part of why: every fresh submission resets to a Gaussian rating at $\mu_0 = 600$, so an early score carries real uncertainty regardless of the agent's true strength. From there I changed how I worked: ship designs that are safe by construction, let live data be the actual judge, and stop leaning too hard on local A/B deltas that don't clear real statistical noise.

I carried the same recipe to a second archetype, Mega Lucario ex, and pushed the training data further with cross-play generation: instead of one policy playing itself (tried earlier and rejected, since it measurably hurt held-out accuracy through what looked like a distillation ceiling), I had several independently-evolved variants of my own agent play each other and kept only the winning side's decisions, mixed with real replay data as a solid, non-diluted anchor. The result split honestly between the two decks. On Mega Lucario ex, held-out accuracy climbed step by step as cross-play was added, 57.3% to 58.5% to 59.3%. On Dragapult it came out flat to slightly negative (63.0% versus 63.5% real-only) despite 40,300 extra games, but it produced a real, twice-replicated 6.0-point live-architecture gain against Mega Lucario ex specifically, which happens to be my own worst matchup (more on that below).

Generalization, not just one good matchup

The scoring criteria explicitly reward avoiding over-reliance on one matchup or one situational edge. Late in the project I ran a deliberately broad verification pass: round-robin testing every current build against every other build I'd ever made, including the rejected ones, then against a widened local reference set covering archetypes I'd never tested against, then against every remaining opponent left in the archive. Roughly 22,000 local games across three escalating passes. It surfaced weaknesses a narrower test would have missed entirely. One candidate build had passed every earlier, narrower A/B, but lost to 8 of 16 sibling Dragapult variants and 2 of 6 Lucario-line builds once I widened the net, so I reverted to its stronger predecessor. That round-robin, not a single flattering matchup number, is the honest generalization evidence behind the current live builds.

I also found and fixed twelve concrete bugs through game-log instrumentation rather than guess-

work. Among them: a Kaggle sandbox crash from an undefined `__file__` reference that local testing couldn't have caught, since Kaggle's sandbox loads agents with `exec()` rather than a normal import; a stadium card offered 2,377 times and played zero, from a play-condition sign that was simply inverted; and a sequencing bug where a strong Supporter card lost its one-per-turn slot to a worse card purely from the order decisions were evaluated in, not from any real decision quality problem.

Deck construction and matchup targeting

I piloted Dreepy/Drakloak/Dragapult ex, a fast Ghost/Dragon evolution attacker, as the primary deck, since it had the highest live ceiling and the most complete evidence trail. Mega Lucario ex was the second deck, and that choice came from data rather than a hunch: mining my own 84 real ladder replays showed it was simultaneously my most common real opponent (22.6% of the field) and my worst matchup project-wide (31.6% win rate). That's a concrete reason to prioritize it over a matchup I merely found annoying to play against.

Two decklist decisions worth walking through in detail:

Card scarcity, not just card choice. Instrumentation showed that 72% of Dragapult's missed Boss's Orders plays came from pure hand scarcity, not from bad decisions once the card was in hand. I cut a card that was genuinely dead weight on my board, a second Team Rocket's Watchtower (it only disables Colorless-type Abilities, and my researched wall matchup is Grass-type), for a fourth Boss's Orders, the legal maximum. The measured effect: the Mega Lucario ex matchup moved from a roughly 47 to 49% baseline up to 56.5%, isolated and reproducible.

A dead card can be a bug, not a design choice. A card-text audit flagged Gravity Mountain, a stadium that's harmless to my own board, as a plausible cut. Instrumentation found something different: the card wasn't dead by design at all. A one-line polarity bug refused to ever play it into an empty stadium slot, so it was offered 2,377 times across 150 games and played exactly zero. Fixing the bug (rather than cutting the card) was the right call. Post-fix it plays about 1.5 times a game and produced a measured 3.2-point gain specifically against my one reference deck that includes a Stage-2 Pokémon, which is exactly where the mechanic should matter. The gain being matchup-selective rather than a blanket shift is what tells me the mechanism, not noise, is what moved.

Every deck change was tested in isolation, at 150 to 499 games per matchup, before being combined with anything else, including one case where I found and fixed a real negative interaction between two independently good fixes before shipping them together.

Results, and what's still open

With the leaderboard now frozen, the final tracked pair reads `dragapult_combined_final_v2` at 590.0 and `lucario_crossplay_reranker_v1` at 554.6, placing me 3829th of 6807 on the public leaderboard, just above the midpoint of the field. The architecture's proven ceiling, 924.2 on `bc_reranker_v2`, remains the project's best live result, and it came from the safe-by-construction reranker design above rather than from deck changes alone; the frozen pair reflects where the newest cross-play retrains happened to land in the final rating window, not a ceiling on the approach itself. I haven't closed the gap to the leaderboard's top tier, and I'd rather say that plainly than round up: `lucario_crossplay_reranker_v1` still has a real, unresolved weakness against Alakazam-shaped decks locally (around 35% win rate) that I chose to ship rather than delay further, and live-score

volatility under the platform's Gaussian rating means any single day's number is a noisy read rather than a verdict.

The lesson I'm taking from this project is a methodological one: I stopped letting local A/B testing be the last word on a design, especially one whose value depends on tie-breaking or plan-level consistency. Those are exactly the properties a small local sample measures worst, and exactly where my biggest win and my biggest false negatives both came from.